

C++ Notes

Lambdas — Items 31 à 34

Adam HAMOUC

May 2026

1. C'est quoi une lambda — et pourquoi s'en servir

Une lambda est une **fonction anonyme définie là où on en a besoin**, sans avoir à lui donner un nom ou créer une fonction séparée ailleurs dans le code. Elle peut **capturer les variables de son scope** — ce qu'une fonction normale ne peut pas faire.

Le compilateur génère automatiquement une classe et l'instancie. **Lambda = code compilé. Closure = l'objet créé à l'exécution.**

```
// Syntaxe : [capture](parametres) { corps }
auto isDead = [](Enemy* e) { return e->health <= 0; };

// Sans lambda : on doit déclarer une fonction separee
bool isDead(Enemy* e) { return e->health <= 0; } // loin du code qui l'utilise

// Avec lambda : definie la ou on en a besoin
auto it = std::remove_if(enemies.begin(), enemies.end(),
    [](Enemy* e) { return e->health <= 0; }
);
```

Utilisations typiques en engine/game dev :

- Tri de draw calls : `std::sort` avec un critere custom (depth, material...).
- Filtrage d'entites : `FilterByPredicate` sur un `TArray`.
- Callbacks de passes de rendu : definir inline ce que fait une `RenderPass`.
- Jobs asynchrones : envoyer un morceau de travail a un thread pool.
- Evenements / delegates : reaktion a un input, une collision, une mort.

```
// Tri front-to-back pour les opaques (optimise le depth test GPU)
std::sort(drawCalls.begin(), drawCalls.end(),
    [](const DrawCall& a, const DrawCall& b) { return a.depth < b.depth; });

// Callback de render pass
renderGraph.addPass("GBuffer", [&](RenderContext& ctx) {
    ctx.bindPipeline(gBufferPipeline);
    ctx.drawMeshes(visibleMeshes);
});

// Job asynchrone
threadPool.submit([mesh = std::move(mesh)]() { mesh->uploadToGPU(); });

// Filtre Unreal
TArray<AActor*> visible = actors.FilterByPredicate(
    [&frustum](const AActor* a) { return frustum.contains(a); });
```

2. La capture — acceder aux variables du scope

La capture permet a la lambda d'utiliser des variables definies **en dehors d'elle**. Sans capture, la lambda ne voit que ses propres parametres.

Deux modes : par valeur `[x]` (copie au moment de la creation) ou par reference `[&x]` (acces direct a la variable originale).

```
float threshold = 50.f;

// Sans capture : threshold est invisible
auto f = [](Enemy* e) { return e->health < threshold; }; // ERREUR

// Par valeur : copie de threshold au moment de la creation de la lambda
auto f = [threshold](Enemy* e) { return e->health < threshold; }; // OK

// Par reference : acces direct, threshold peut changer entre-temps
auto f = [&threshold](Enemy* e) { return e->health < threshold; }; // OK mais risque
```

Pieges des captures par default [=] et [&]

Eviter les captures par default. Elles masquent ce qui est vraiment capture et creent des bugs difficiles a diagnostiquer.

Capture dans une methode de classe : [=] ne capture pas les membres — il capture **this** (le pointeur). Si l'objet est detruit avant l'appel de la lambda → crash.

```
class Widget {
    int divisor = 5;
    void addFilter() {
        // DANGER : [=] capture this, pas divisor
        // Si Widget detruit avant appel de la lambda => dangling pointer => crash
        filters.push_back([=](int v) { return v % divisor == 0; });

        // CORRECT : capturer la valeur explicitement avec init capture
        filters.push_back([div = divisor](int v) { return v % div == 0; });
        // div est une copie independante, vit dans la closure, Widget peut mourir
    }
};
```

Resume des captures :

- [x] par valeur : copie au moment de la creation. Safe si l'original meurt.
- [&x] par reference : acces direct. Dangereux si la variable sort du scope.
- [=] dans une methode : capture **this**, pas les membres — piegeux.
- [div = divisor] init capture : copie explicite d'un membre — recommande.
- [*this] (C++17) : copie complete de l'objet — safe mais couteux.

3. Init capture — move dans une closure (C++14)

En C++11, impossible de **deplacer** un objet dans une closure. Problematique pour les `unique_ptr` (non copiables) ou les gros containers (copie trop couteuse).

C++14 introduit l'**init capture** : [nom = expression] — on nomme soi-meme le membre de la closure et on l'initialise avec n'importe quelle expression, y compris un `std::move`.

```
auto mesh = std::make_unique<RenderMesh>(vertices, indices);

// C++11 : IMPOSSIBLE de transferer ownership dans la closure
// [&mesh] : dangereux si mesh sort du scope
// [mesh] : tente une copie => unique_ptr non copiable => ERREUR

// C++14 : init capture => ownership transfere dans la closure
//      membre de la closure      expression d'init
auto job = [m = std::move(mesh)]() { m->draw(); };
// mesh est vide ici. m vit dans la closure, detruite avec elle.

// Ou directement, sans variable intermediaire
auto job = [m = std::make_unique<RenderMesh>(vertices, indices)]()
    { m->draw(); };
```

Emulation C++11 via std::bind (si C++14 indisponible) :

```
// std::bind move-construct ses arguments rvalue dans un bind object
// La lambda recoit une reference vers cet objet interne
auto job = std::bind(
    [](const std::unique_ptr<RenderMesh>& m) { m->draw(); },
    std::move(mesh) // move-construct dans le bind object
);
// Equivalent fonctionnel mais beaucoup plus verbeux
```

4. Perfect forwarding dans une lambda generique (C++14)

Une lambda avec `auto&&` est une **lambda generique** — le compilateur genere un `operator()` templatise. Pour forwarder parfaitement le parametre, on ne peut pas ecrire `std::forward<T>` (pas de T disponible). Solution : `std::forward<decltype(param)>(param)`.

```
// Mauvais : x toujours passe comme lvalue a normalize
auto f = [](auto x) { return func(normalize(x)); };

// Correct : universal reference + forward via decltype
auto f = [](auto&& param) {
    return func(normalize(std::forward<decltype(param)>(param)));
};

// Variadic : accepte n'importe quel nombre de parametres
auto f = [](auto&&... params) {
    return func(normalize(std::forward<decltype(params)>(params)...));
};
```

5. Lambda & std::bind — toujours en C++14

`std::bind` produit des objets fonctions (bind objects) en liant des arguments a un callable. En C++14, les lambdas remplacent `std::bind` dans tous les cas : plus lisibles, plus expressives, et potentiellement plus rapides (inlining).

```
// BIND : opaque. Quand est evalue now() ? Que fait _1 ? Quel setAlarm ?
auto f = std::bind(setAlarm,
    std::bind(std::plus<>(), steady_clock::now(), 1h), _1, 30s);

// LAMBDA : se lit comme du code normal
auto f = [](Sound s) {
    setAlarm(steady_clock::now() + 1h, s, 30s); // now() evalue a l'appel
};
// Si setAlarm est surchargee => lambda choisit automatiquement la bonne version
// bind => erreur de compilation, ambiguïté sur le nom
```

std::bind justifie uniquement en C++11 pour :

- Emuler le move capture (remplace par init capture en C++14).
- Objets polymorphiques avec `operator()` templatisé (remplace par `auto param`).

En C++14 : **toujours utiliser une lambda.**

Recap global

- Lambda = syntaxe. Closure = objet genere a l'execution. Les deux sont inséparables.
- Capturer explicitement, jamais [=] ou [&] par default.
- Dans une methode : [=] capture `this`, pas les membres. Utiliser [`x = membre`].
- `unique_ptr` ou gros container dans une closure → [`x = std::move(x)`] (C++14).
- Perfect forwarding dans une lambda generique → `std::forward<decltype(p)>(p)`.
- Lambda > `std::bind` en C++11. Lambda ≫ `std::bind` en C++14.